

Neuro-Adaptive Control Without State Measurement: A 1D-CNN Framework for Higher-Order Systems Using Historical Output Data

Myeongseok Ryu, Naol Samuel Erega, and Kyunghwan Choi

Abstract—This study proposes a convolutional neural network (CNN)-based adaptive controller with three notable features: 1) it determines control input directly from historical sensor data; 2) it learns the desired control policy during real-time implementation without using a pre-trained network; and 3) the asymptotic tracking error convergence is proven during the learning process. An adaptive law for learning the desired control policy is derived using the gradient descent optimization method, and its stability is analyzed based on the Lyapunov approach. A simulation study using a control-affine nonlinear system demonstrated that the proposed controller exhibits these features, and its performance can be tuned by manipulating the design parameters. In addition, it is shown that the proposed controller has a superior tracking performance to that of a deep neural network (DNN)-based adaptive controller.

Index Terms—Neuro-adaptive control, convolutional neural network (CNN), ...

NOTATION

In this study, the following notation is used:

- $\mathbb{R}^n$ denotes the n -dimensional Euclidean space.
- $\mathbb{R}^{n \times m}$ denotes the set of $n \times m$ real matrices.
- A vector and a matrix are denoted by $x = [x_i] \in \mathbb{R}^n$ and $A := [a_{ij}] \in \mathbb{R}^{n \times m}$, respectively.
- I_n denotes the $n \times n$ identity matrix, and $0_{n \times m}$ denotes the $n \times m$ zero matrix.
- $\text{vec}(A)$ denotes the vectorization of the matrix A .
- A matrix $P \succ 0$ ($P \succeq 0$) is positive definite (positive semidefinite).
- $\lambda_{\min}(A)$ denotes the minimum eigenvalue of the matrix $A \in \mathbb{R}^{n \times n}$.
- $\otimes$ denotes the Kronecker product [1].

I. INTRODUCTION

This document is a model and instructions for LATEX. Please observe the conference page limits. For more information about how to become an IEEE Conference author or how to write your paper, please visit IEEE Conference Author Center website: <https://conferences.ieeeauthorcenter.ieee.org/>.

Myeongseok Ryu, Naol Samuel Erega and Kyunghwan Choi are with the Cho Chun Shik Graduate School of Mobility, Korea Advanced Institute of Science and Technology (KAIST), Daejeon 34051, Korea (e-mail: dding_98@kaist.ac.kr, samuelnaol7@kaist.ac.kr and kh.choi@kaist.ac.kr).

[MS: Should be reviewed] This work was supported in part by a Korea Research Institute for Defense Technology planning and advancement (KRIT) grant funded by Korea government DAPA (Defense Acquisition Program Administration) (No. KRIT-CT-22-087, Synthetic Battlefield Environment based Integrated Combat Training Platform) and in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2025- 00554087). (Corresponding author: Kyunghwan Choi)

A. Motivation

The IEEEtran class file is used to format your paper and style the text. All margins, column widths, line spaces, and text fonts are prescribed; please do not alter them. You may note peculiarities. For example, the head margin measures proportionately more than is customary. This measurement and others are deliberate, using specifications that anticipate your paper as one part of the entire proceedings, and not as an independent document. Please do not revise any of the current designations.

B. Contributions

The IEEEtran class file is used to format your paper and style the text. All margins, column widths, line spaces, and text fonts are prescribed; please do not alter them. You may note peculiarities. For example, the head margin measures proportionately more than is customary. This measurement and others are deliberate, using specifications that anticipate your paper as one part of the entire proceedings, and not as an independent document. Please do not revise any of the current designations.

C. Organization

The IEEEtran class file is used to format your paper and style the text. All margins, column widths, line spaces, and text fonts are prescribed; please do not alter them. You may note peculiarities. For example, the head margin measures proportionately more than is customary. This measurement and others are deliberate, using specifications that anticipate your paper as one part of the entire proceedings, and not as an independent document. Please do not revise any of the current designations.

II. PROBLEM FORMULATION AND CONTROL OBJECTIVES

A. Control Objectives

- end to end controller for path tracking
- we expect 1D CNN operation to extract underlying dynamics from stacked input(i.e. error derivatives, integrals, etc.) and provide informative features for steering control without an explicit information.
- we expect the controller adapts the vehicle parameters changes online(eg. tire characteristics, vehicle mass, etc.)

- we expect controller adapt its parameters online to handle the changing dynamics and maintain good tracking performance across different path segments (e.g., straight vs. curved) and varying conditions (e.g., different speeds, road friction, etc.).

III. SYSTEM DETAILS

A. CVL-CONAC Architecture

The network $\hat{\Phi}$ is designed as a hybrid architecture suitable for capturing spatio-temporal dynamics from historical data[??].

1) *Convolutional Layers (CVLs)*: The CVLs are represented recursively as:

$$\Phi_{j_c}^C = \begin{cases} O(\phi_{j_c}^C(\Phi_{j_c-1}^C), \Omega_{j_c}, B_{j_c}), & j_c \in [1, \dots, k_c] \\ O(\phi_0^C(X), \Omega_0, B_0), & j_c = 0 \end{cases}, \quad (1)$$

where $X \in \mathbb{R}^{n_0 \times m_0}$ denotes the network input matrix, $O : \mathbb{R}^{n_{j_c} \times m_{j_c}} \times \mathbb{R}^{p_{j_c} \times m_{j_c} \times q_{j_c}} \times \mathbb{R}^{q_{j_c}} \rightarrow \mathbb{R}^{n_{j_c+1} \times m_{j_c+1}}$ denotes the CNN operator (see Appendix A), and $\phi_{j_c}^C : \mathbb{R}^{n_{j_c} \times m_{j_c}} \rightarrow \mathbb{R}^{n_{j_c} \times m_{j_c}}$ denotes the matrix activation function (i.e., $\phi_{j_c}^C(\Phi_{j_c-1}^C)_{(i,j)} = \sigma_{j_c}(\Phi_{j_c-1}^C)_{(i,j)}$) for some activation function $\sigma_{j_c} : \mathbb{R} \rightarrow \mathbb{R}$. The first activation function ϕ_0^C should be a bounded nonlinear function to ensure that the input to the first CNN operator is bounded. In this study, $\alpha_1 \tanh(\cdot)$ is selected with $\alpha_1 \in \mathbb{R}_{>0}$. The filter set Ω_{j_c} contains q_{j_c} filters $W_{j_c}^{(i)} \in \mathbb{R}^{p_{j_c} \times m_{j_c}}, \forall i \in [1, \dots, q_{j_c}]$, and is represented as $\Omega_{j_c} = \{W_{j_c}^{(1)}, \dots, W_{j_c}^{(q_{j_c})}\}$, where superscript i denotes the filter index. The bias vector B_{j_c} consists of q_{j_c} biases.

The weights for the j_c -th CVL layer are collectively represented as $\tilde{H}_{C_{j_c}} \triangleq [\text{vec}(\Omega_{j_c})^T, B_{j_c}^T]^T$.

2) *Fully-Connected Layers (FCLs)*: The output matrix of the CVLs is input to the concatenate layer $C(\Phi_{k_c}^C) = [\text{vec}(\Phi_{k_c}^C)^T, 1]^T$ before the input layer of FCLs for compatibility between the CVLs and FCLs. The FCLs are represented recursively as:

$$\Phi_{j_f}^F = \begin{cases} V_{j_f}^T \phi_{j_f}^F(\Phi_{j_f-1}^F), & j_f \in [1, \dots, k_f] \\ V_0^T C(\Phi_{k_c}^C), & j_f = 0 \end{cases}, \quad (2)$$

where $V_{j_f} \in \mathbb{R}^{l_{j_f}+1 \times l_{j_f}+1}$ denotes the weight matrix, $\phi_{j_f}^F : \mathbb{R}^{l_{j_f}} \rightarrow \mathbb{R}^{l_{j_f}+1}$ denotes the vector activation function defined by $\phi_{j_f}^F(\Phi_{j_f-1}^F) \triangleq [\sigma_{j_f}(\Phi_{j_f-1}^F)^T, 1]^T$ for $j_f \in [1, \dots, k_f - 1]$, for some nonlinear activation functions $\sigma_{j_f} : \mathbb{R} \rightarrow \mathbb{R}$ such as $\tanh(\cdot)$. Note that $C(\Phi_{k_c}^C)$ and $\Phi_{j_f}^F$ are augmented by 1 to consider the bias of the preceding input layer as a weight in the weight matrix. The output of the FCLs is the final output of the CNN architecture and is represented as $\Phi \equiv \Phi_{k_f}^F$.

The FCL weights for the j_f -th layer are $\tilde{H}_{F_{j_f}} \triangleq \text{vec}(V_{j_f})$.

IV. ADAPTATION LAWS

A. Optimization Problem

V. STABILITY ANALYSIS

VI. NUMERICAL VALIDATION

A. Validation Setup

To validate the proposed controller, a numerical simulation is conducted using a nonlinear vehicle model with four-wheel

steering and actuator lag. The following 2-DOF vehicle lateral dynamics [2] are considered:

$$\begin{cases} \frac{d}{dt} v_y = \frac{1}{m} (F_f + F_r) - v_x r, \\ \frac{d}{dt} r = \frac{1}{I_z} (F_f l_f - F_r l_r), \end{cases} \quad (3)$$

where v_x is the longitudinal velocity, v_y is the lateral velocity, and $r = \frac{d}{dt} \psi$ is the yaw rate, m is the vehicle mass and I_z is the yaw moment of inertia, l_f and l_r are the distances from the center of gravity to the front and rear axles, respectively, and F_f and F_r are the front and rear lateral tire forces, respectively, and modelled as:

$$F_i = F_{\max,i} \tanh\left(\frac{C_i}{F_{\max,i}} \alpha_i\right), \quad \forall i \in \{f, r\}, \quad (4)$$

where C_i , $F_{\max,i}$, and α_i are the cornering stiffness, saturation limit, and slip angle of the front ($i = f$) and rear ($i = r$) tires, respectively. The slip angles are given by

$$\alpha_f = \delta_f - \frac{v_y + l_f r}{v_x}, \quad \alpha_r = \delta_r - \frac{v_y - l_r r}{v_x}, \quad (5)$$

where δ_f and δ_r are the effective front and rear steering angles respectively. In order to capture the lag between the steering commands and the actual steering angles, the actuator dynamics are modeled as: $\frac{d}{dt} \delta_i = \frac{\omega_i - \delta_i}{\tau}$, $\forall i \in \{f, r\}$, where ω_i , $\forall i \in \{f, r\}$ denotes the front and rear steering commands, respectively, and $\tau > 0$ is the actuator time constant.

The global position kinematics are given by (6):

$$\dot{X}_a = v_x \cos \psi - v_y \sin \psi, \quad \dot{Y}_a = v_x \sin \psi + v_y \cos \psi, \quad (6)$$

where (X_a, Y_a) denotes the vehicle position in the global frame, and ψ denotes the yaw angle. The yaw rate $\dot{\psi}$ is obtained by integrating the yaw acceleration $\ddot{\psi}$ in (3).

In this section, the proposed controller is validated through numerical simulation using the nonlinear 4WS vehicle model described in Section II. The desired trajectory is selected as a closed stadium-shaped path composed of two straight segments and two semicircular arcs. Let L and R denote the straight length and turning radius, respectively. Then, the lower and upper straight segments are defined by

$$\mathcal{P}_1 = \{(x, y) \mid 0 \leq x \leq L, y = 0\}, \quad (7)$$

$$\mathcal{P}_2 = \{(x, y) \mid 0 \leq x \leq L, y = 2R\}, \quad (8)$$

The left and right turning sections are semicircles centered at $C_L = (0, R)$ and $C_R = (L, R)$, respectively. This path contains both zero-curvature and constant-curvature segments and is therefore suitable for evaluating straight-line regulation and turning performance within a single experiment. Figure 1 illustrates the stadium path together with the vehicle pose, body-fixed frame, and tracking error used in the validation setup.

To generate the path-following reference at each simulation step, the current vehicle position (X_a, Y_a) is projected onto the corresponding path segment. For straight segments, the desired heading is:

$$\psi_{\text{des}} = \begin{cases} 0, & \text{lower straight,} \\ \pi, & \text{upper straight,} \end{cases} \quad (9)$$

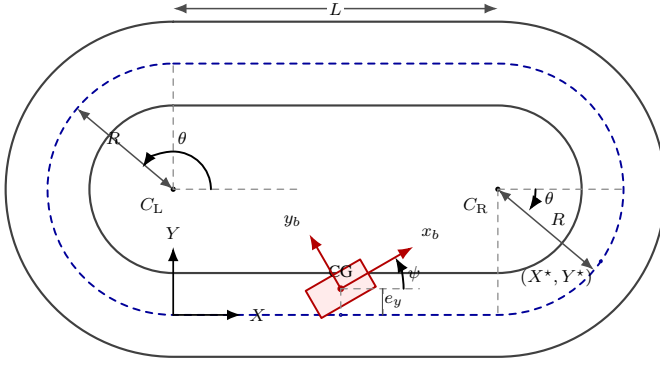


Fig. 1: Stadium-track geometry and vehicle pose used in the validation setup.

For the arc segments, the projected point is computed by

$$X^* = x_c + R \cos \theta, \quad Y^* = y_c + R \sin \theta, \quad (10)$$

with

$$\theta = \text{atan2}(Y_a - y_c, X_a - x_c), \quad (11)$$

The desired heading is then taken as the tangent direction of the arc,

$$\psi_{\text{des}} = \theta + \frac{\pi}{2}, \quad (12)$$

The tracking errors are then defined by the lateral displacement from the projected point and the heading error.

B. Validation Results

The validation results are presented in terms of tracking errors, steering commands, and the global vehicle trajectory. In particular, the lateral and heading errors are used to assess whether the proposed 1D CNN-MLP controller can regulate the vehicle toward the desired path while adapting its parameters online. The front and rear steering-rate commands illustrate the control effort generated by the end-to-end policy, and the global XY trajectory provides a direct geometric comparison between the desired path and the actual motion.

In the final manuscript, this subsection should be organized around the simulation plots produced by the current MATLAB implementation. A compact presentation consistent with the reference paper would first discuss the validation conditions, then summarize the tracking behavior in the time domain, and finally comment on the overall path-following accuracy and adaptation behavior. Root-mean-square tracking errors and selected controller settings can also be reported in a table to make the comparison more concise.

VII. CONCLUSION

REFERENCES

- [1] D. S. Bernstein and D. S. Bernstein, *Matrix Mathematics: Theory, Facts, and Formulas (Second Edition)*. Princeton: Princeton University Press, 2009.
- [2] R. Rajamani, *Vehicle dynamics and control*, 2nd ed., ser. Mechanical Engineering Series. New York, NY: Springer, Dec. 2012.



Myeongseok Ryu received the B.S. degree in Mechanical Engineering from Incheon National University, South Korea, in 2023, and the M.S. degree in Mechanical Engineering from GIST, South Korea, in 2025. He is currently pursuing the Ph.D. degree in the Cho Chun Shik Graduate School of Mobility at Korea Advanced Institute of Science and Technology (KAIST), South Korea. His research interests include neuro-adaptive control, online optimization, and contraction theory.



Naol Samuel Erega [MS: Should be reviewed] received his B.S. degree in Physics and Engineering Physics from Yonsei University, Wonju, South Korea, in 2023. He is currently pursuing the M.S. degree in Mechanical Engineering from GIST, Gwangju, South Korea. His research interests include robot control, neural-networks, and model identification.



Kyunghwan Choi received his B.S., M.S., and Ph.D. degrees in Mechanical Engineering from KAIST, Daejeon, South Korea, in 2014, 2016, and 2020, respectively. Following his Ph.D., he worked at the Center for Eco-Friendly & Smart Vehicles at KAIST as a Postdoctoral Fellow and later as a Research Assistant Professor. In 2022, he joined GIST as an Assistant Professor in the Department of Mechanical and Robotics Engineering. Currently, he serves as an Assistant Professor at the Cho Chun Shik Graduate School of Mobility at KAIST, where he is also the Director of the Mobility Intelligence and Control Laboratory. His research focuses on optimal and learning-based control for connected, automated, and electrified vehicles (CAEVs).